



Aqueduct Materials (aqueducts)

Aquaducten worden gemaakt door een van de N Romeinse materialen te gebruiken. Het i -de materiaal heeft een sterkte S_i en een prijs P_i .

We zeggen dat het i -de materiaal het j -de materiaal *domineert* dan en slechts dan als:

- het is sterker is ($S_i > S_j$), en
- het is goedkoper is ($P_i < P_j$).

De Senaat wil **een** materiaal beperken dat exclusief in Rome gebruikt wordt, om er zeker van te zijn dat de stad een uniek uiterlijk behoudt. De $N - 1$ andere materialen zullen *beschikbaar* zijn voor de rest van het keizerrijk.

Een materiaal is *optimaal* voor de rest van het keizerrijk als

- het beschikbaar is, en
- het niet gedomineerd wordt door een ander beschikbaar materiaal.

Voor elke k , stel dat het materiaal gereserveerd voor Rome k is, tel het aantal C_k van optimale materialen voor de rest van het keizerrijk.

Implementatie

Je moet één .cpp bronbestand inleveren.



In de bijlagen van deze taak vind je een sjabloon `aqueducts.cpp` met een voorbeeld implementatie.

Je moet de volgende functie implementeren:

C++

```
vector<int> count(int N, vector<int> S, vector<int> P)
```

- Integer N staat voor het aantal materialen.
- De array S , genummerd van 0 tot en met $N - 1$, bevat de waarden $S_0, S_1, \dots, S_{N-1}$.
- De array P , genummerd van 0 tot en met $N - 1$, bevat de waarden $P_0, P_1, \dots, P_{N-1}$.
- De functie moet een array C teruggeven, met daarin de waardes $C_0, C_1, \dots, C_{N-1}$ waar C_k het aantal optimale materialen dat over is voor de rest van het keizerrijk als het materiaal k gereserveerd is voor Rome.

Voorbeeld Grader

In de bijlagen van deze taak vind je een versimpelde versie van de grader die gebruikt wordt tijdens de evaluatie, die je kunt gebruiken om je oplossingen lokaal te testen. De voorbeeld grader leest de invoer van `stdin`, roept de functie `count` aan en schrijft uitvoer naar `stdout` in het volgende formaat.

De invoer bestaat uit $N + 1$ regels, waaronder:

- Regel 1: de integer N .
- Regel $2 + i$ ($0 \leq i < N$): de integers S_i en P_i .

De uitvoer bestaat uit één regel, met de waardes die door de functie `count` zijn teruggegeven.

Randvoorwaarden

- $2 \leq N \leq 300\,000$.

- $1 \leq S_i, P_i \leq 1\,000\,000$.
- Voor elke $0 \leq i < j \leq N - 1$, hebben we $S_i \neq S_j$ en $P_i \neq P_j$.

Scoring

Je programma zal getest worden met meerdere testgevallen gegroepeerd in subtaken. Om de score voor een subtaak te krijgen moet je programma alle testgevallen daarvan correct oplossen.

- **Subtask 0 [0 punten]:** Voorbeeld testgevallen.
- **Subtask 1 [13 punten]:** $N \leq 100$.
- **Subtask 2 [22 punten]:** $N \leq 5000$.
- **Subtask 3 [28 punten]:** $|S_i - P_i| \leq 2$.
- **Subtask 4 [37 punten]:** Geen extra randvoorwaarden.

Voorbeelden

stdin	stdout
3 10 20 30 10 20 30	1 2 1
9 10 11 4 6 11 13 14 7 1 4 6 10 9 2 12 12 3 5	2 2 2 3 2 2 4 2 2

Uitleg

Uitleg van het eerste voorbeeld.

- Als $k = 0$, is materiaal 1 optimaal.
- Als $k = 1$, zijn materialen 0 en 2 optimaal.
- Als $k = 2$, is materiaal 1 optimaal.

Uitleg van het tweede voorbeeld.

- Als $k = 0$, zijn materialen 3 en 6 optimaal.
- Als $k = 1$, zijn materialen 3 en 6 optimaal.
- Als $k = 2$, zijn materialen 3 en 6 optimaal.
- Als $k = 3$, zijn materialen 0, 6 en 7 optimaal.
- Als $k = 4$, zijn materialen 3 en 6 optimaal.
- Als $k = 5$, zijn materialen 3 en 6 optimaal.
- Als $k = 6$, zijn materialen 1, 3, 4 en 8 optimaal.
- Als $k = 7$, zijn materialen 3 en 6 optimaal.
- Als $k = 8$, zijn materialen 3 en 6 optimaal.